

ANDROID STUDIO

ACTIVITY

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Zer da Activity bat?
2. Zer da Layout bat?
3. Activity baten zatiak Android-en
4. Zer da Activity bat Android-en?
5. Android manifest
6. Programazio kontzeptuak
7. Programazioa
8. Main_Activity
9. Activity baten bizi-zikloa

ZER DA ACTIVITY BAT?

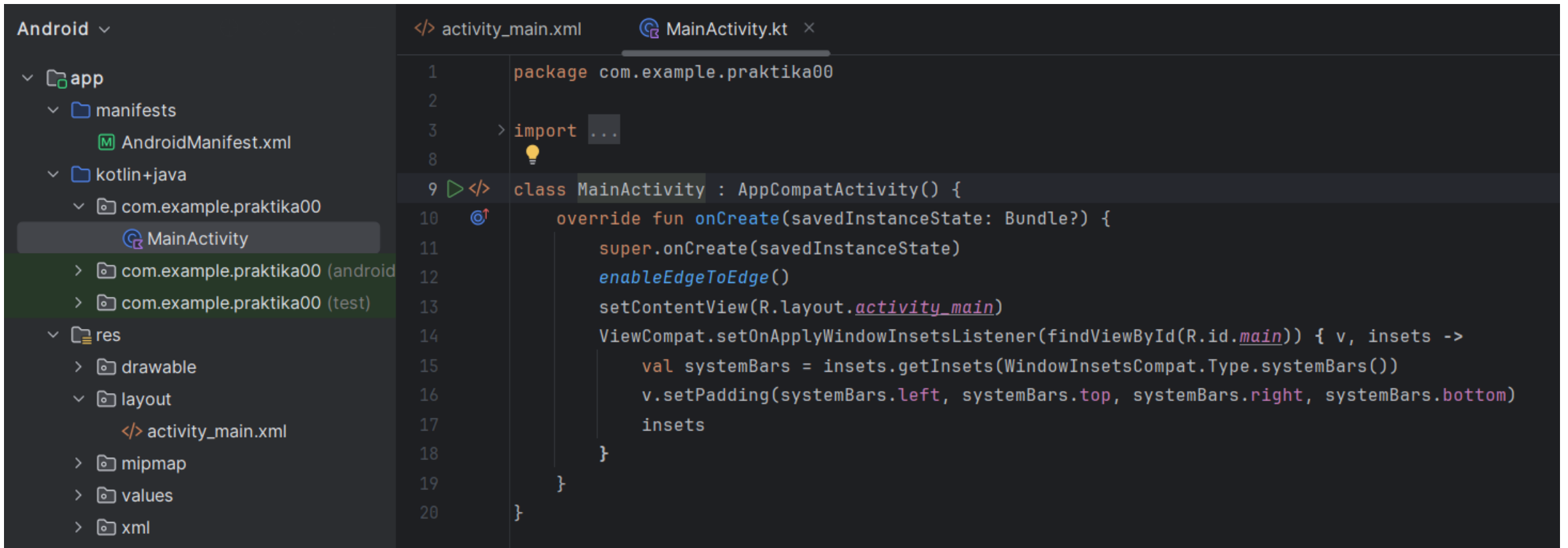
Gure lehen jarduera, **MainActivity**.

- ▶ Activity bat erabiltzaileak ikusten duen pantaila bakoitza da, eta bertan adierazten dira metodoak (egin daitekeena). Adibidez, demagun aplikazioa sukaldekoa dela, activity bat barazkiak moztea, irakitea, frijitzea eta abar izango da.
- ▶ Activity-ek bi zati dituzte, **logikoa** eta **ikusizkoa** (**layout-ak**)
- ▶ Zati logikoan, Kotlinen garatuko duguna, erabiltzailearen interakzio bakoitzari erantzuna emango diogu. Aurreko adibidearekin jarraituz gero, imajina dezagun erabiltzaileak frijitzeko botoia sakatzen duela gure klasera iristeko, eta han ekintza exekutatu da.

ZER DA ACTIVITY BAT?

- ▶ Activity, funtsean, AppCompatActivity zabaltzen duen kotlin (.kt) klasea da, Androidekin komunikatzeko behar diren metodoak dituenena. Beraz, beti izango dugu OnCreate funtzioa, gure jarduera sortzeko eta layout bat esleitzeko ardura izango duena.

ZER DA ACTIVITY BAT?

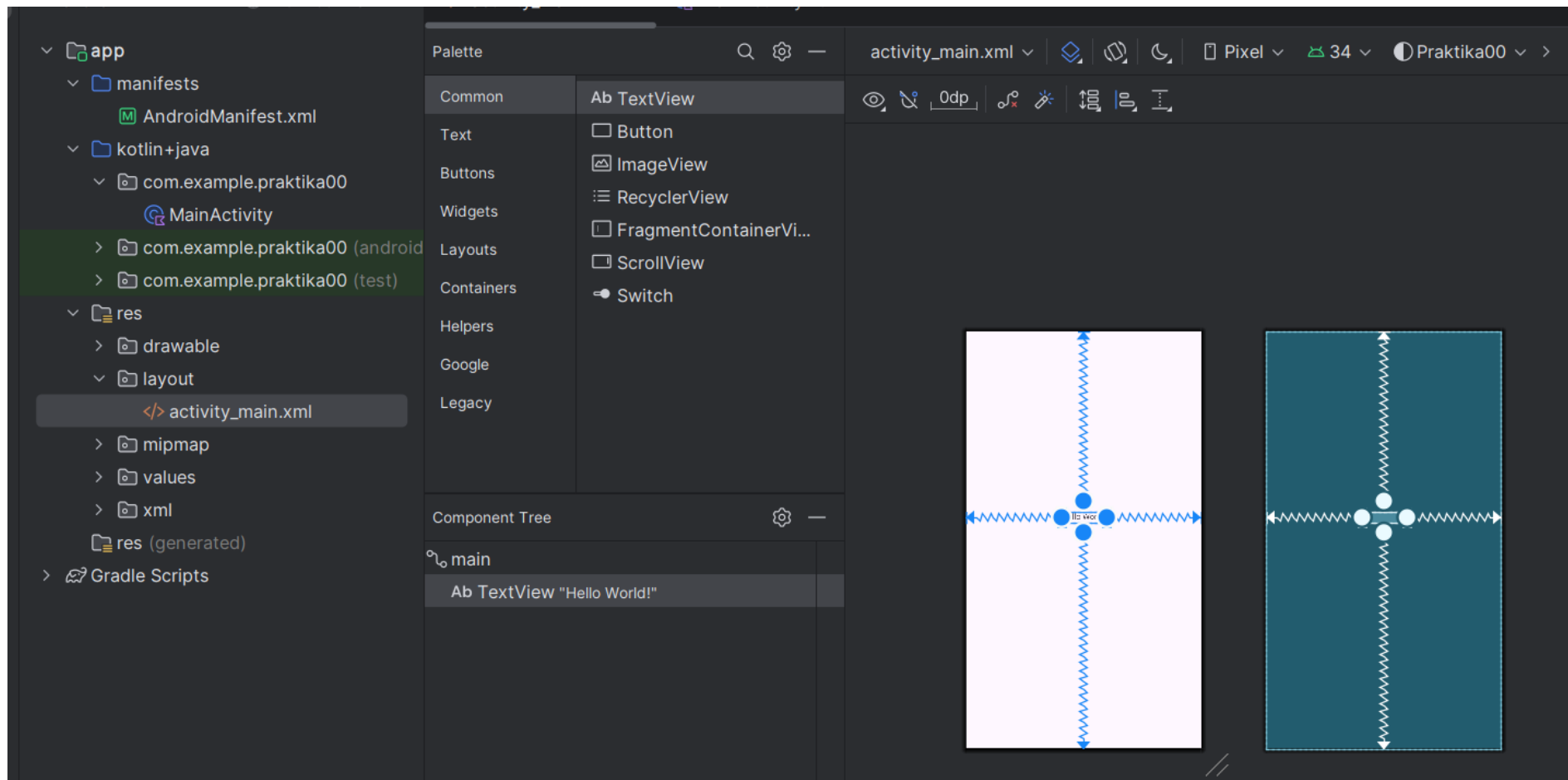


```
Android ▾  
  ▾ app  
    ▾ manifests  
      AndroidManifest.xml  
    ▾ kotlin+java  
      ▾ com.example.praktika00  
        MainActivity  
      ▾ com.example.praktika00 (android)  
      ▾ com.example.praktika00 (test)  
    ▾ res  
      ▾ drawable  
      ▾ layout  
        activity_main.xml  
      ▾ mipmap  
      ▾ values  
      ▾ xml  
  ▾ activity_main.xml  
  MainActivity.kt ×  
1 package com.example.praktika00  
2  
3 > import ...  
8  
9 class MainActivity : AppCompatActivity() {  
10     override fun onCreate(savedInstanceState: Bundle?) {  
11         super.onCreate(savedInstanceState)  
12         enableEdgeToEdge()  
13         setContentView(R.layout.activity_main)  
14         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->  
15             val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())  
16             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)  
17             insets  
18         }  
19     }  
20 }
```

ZER DA LAYOUT BAT?

- ▶ Layout-a jardueraren bigarren zatia da, interfazea.
- ▶ Bertan, osagaiak gehituko dira, hala nola botoiak, irudiak eta abar.
- ▶ Fitxategi horiek oso sinpleak dira, eta osagaiak eramanez edo XMLn gehituz osa daitezke.

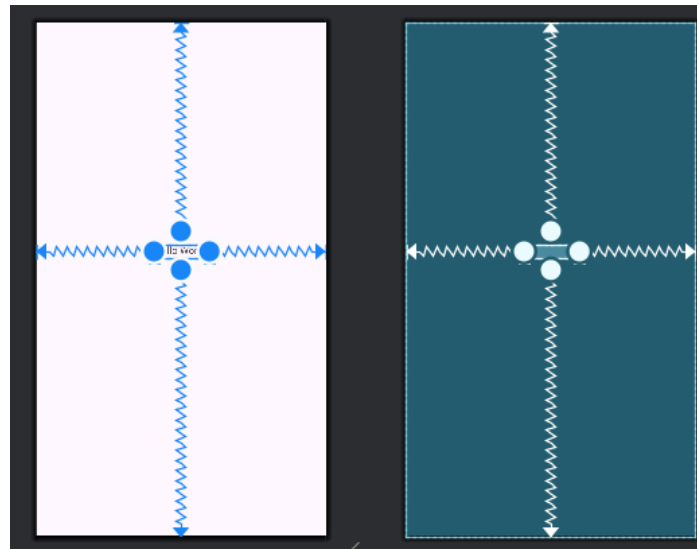
ZER DA LAYOUT BAT?



ACTIVITY BATEN ZATIAK ANDROID-EN

- ▶ Beraz, proiektua sortzen dugunean, lehenik eta behin jarduera(activity) bat sortzen zitzaigun, **MainActivity**.
- ▶ MainActivity hori bi zati izango ditu: klase bat eta xml fitxategi bat.

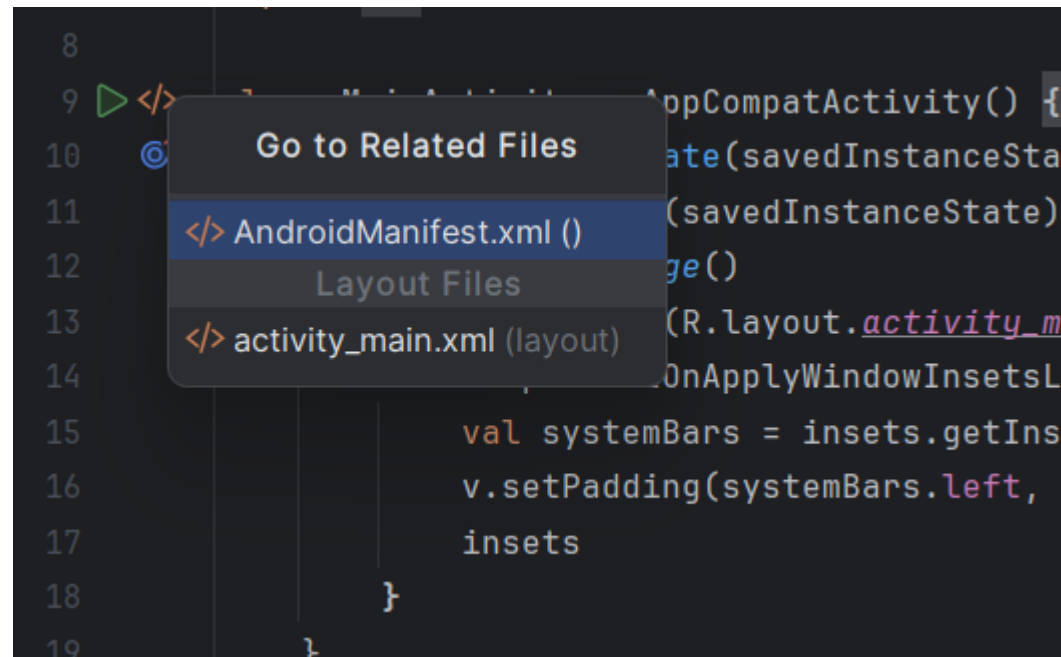
```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContentView(R.layout.activity_main)  
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->  
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())  
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)  
            insets  
        }  
    }  
}
```



```
1 <?xml version="1.0" encoding="utf-8"?>  
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"  
3     xmlns:app="http://schemas.android.com/apk/res-auto"  
4     xmlns:tools="http://schemas.android.com/tools"  
5     android:id="@+id/main"  
6     android:layout_width="match_parent"  
7     android:layout_height="match_parent"  
8     tools:context=".MainActivity">  
9  
10    <TextView  
11        android:layout_width="wrap_content"  
12        android:layout_height="wrap_content"  
13        android:text="Hello World!"  
14        app:layout_constraintBottom_toBottomOf="parent"  
15        app:layout_constraintEnd_toEndOf="parent"  
16        app:layout_constraintStart_toStartOf="parent"  
17        app:layout_constraintTop_toTopOf="parent"  
18        app:layout_constraintVertical_bias="0.442" />  
19  
20 </androidx.constraintlayout.widget.ConstraintLayout>
```

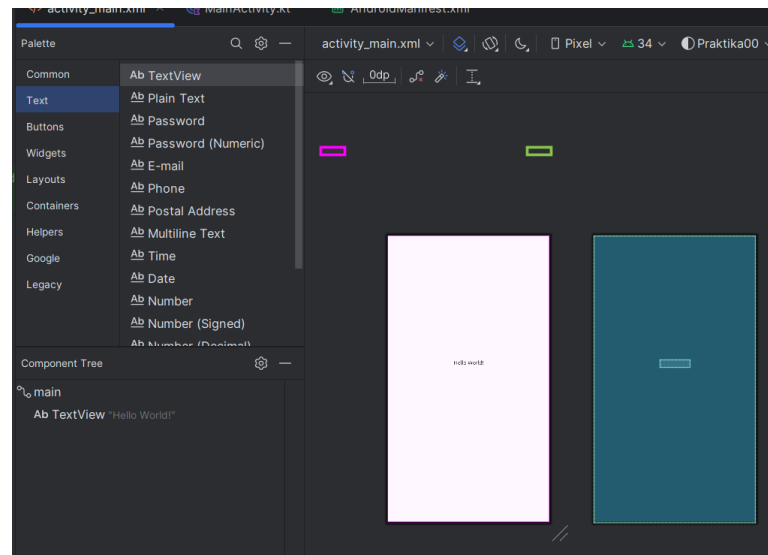

ACTIVITY BATEN ZATIAK ANDROID-EN

- MainActivity-n egonda </> irudiari klikatu eskero xml fitxategietara joako aukera emango digu.



ZER DA ACTIVITY BAT ANDROID-EN?

- ▶ Irudi honetan, pantailaren osagaiak baino ez ditugu; oraingoz, **textview** hau baino ez dugu.
- ▶ Objeto batzuk ditugu, pantailara eraman ditzazkegunak, hala nola testuak, botoiak, irudiak, eta, gero, haiekin gauzak egiten joan gaitezke.



ANDROID MANIFEST

- ▶ Android Manifest, gure aplikazioak egiten duenaren maila altuko konfigurazioa da eta honelako gauzak defini daitezke:
- ▶ **label**: aplikazioaren izena identifikatzen dugu
- ▶ **icon**: erabiliko den ikonoa identifikatzen dugu, defektuz sortzen da besterik adierazi ezean
- ▶ **theme**: gure aplikazioari itxura ematen dion gaia.

ANDROID MANIFEST

- ▶ Adierazi behar diogu zein pantaila erabiliko diren proiektuan, jarduerak(activity)
- ▶ Jarduera hemen gehitzen ez baditugu, ez dugu modurik izango gure proiektuan erabiltzeko.
- ▶ Jarduera Nagusia zein den ere adierazi beharko da, abiarazteko botoiari ematean bera izango baita lehenengo ikusiko dena.

ANDROID MANIFEST

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="Praktika00"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Praktika00"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

PROGRAMAZIO KONTZEPTUAK

- ▶ **Klase** bat elementu baten irudikapena izango da.
- ▶ Klase batek bi gauza izan ditzake: **funtzioak** (ekintza bat gauzatzeko kodezatiak) edo **egoera** (klase horrek gorde ditzakeen balioak dira, gero haiekin gauzak egiteko).
- ▶ Klase bakarretik **objektu** kopuru mugagabea sor daiteke. Normalean, objektu horiei klase baten **instantzia** espezifikoa deitzen zaie, eta beraz, objektuek egoera desberdinak izan ditzakete.

PROGRAMAZIO KONTZEPTUAK

Adibidez: **persona** **klase** mota bat dut, eta pertsona klase horrek bi atributo izango ditu: izen eta adin bat.

- ▶ Guk hainbat objektu izan genitzake, esaterako Jon izena eta 20 urte duen pertsona bat , eta beste bat Tom izena eta 15 urte duena. Bi pertsona, bi objektu, mota honetako bi instantzia eta horietako bakoitza pertsonak balio desberdina du egoera desberdin batean.
- ▶ Bi pertsona ditugu, bi objektu, klase horretako bi instantzia, eta pertsona bakoitzak balio desberdina du, hau da, egoera desberdina.
- ▶ Beraz, egoera desberdinak dituzten objektu horiekin eragiketak egiten ditugunean, eragiketek emaitza desberdinak itzul ditzakete.

PROGRAMAZIO KONTZEPTUAK

```
class Person(val name: String, val age: Int)

val o1 = Person(name: "John", age: 20)
val o2 = Person(name: "Tom", age: 15)

fun test() {
    o1.name
}
```


PROGRAMAZIA

- ▶ **Aldagai** bat aldi baterako balio bat gordetzen duen elementu bat da. Adibidez, guk o1 eta o2 aldagaiak sortu ditugu. Ondoren, aldagai horiek erabil daitezke, adibidez, funtzio batean, o1.name egin dezakegu izena berreskuratzeke, objektu bat sortzeko, bi balio gehitzeko, edo beste eragiketa bat egiteko eta emaitza aldagai horretan gordetzeko.
- ▶ **Funtzioa**: funtzio horren izenari deituz exekutatu ditzakegun kode-multzoak dira. Horrela, gauza konplexuagoak funtzio bakar batean gorde ditzakegu, eta, gainera, hainbat tokitatik dei diezaiokegu, eta kodearen zati hori berrerabil dezakegu.

PROGRAMAZIA

- ▶ **Main_Activity** beste klase batetik hedatzen da(AppCompatActivity). Klaseek beste klase bati buruz dugun ezagutza guztia har dezakete, eta haien alde erabili, guraso-klasearen kode osoa errepikatu behar ez izateko; **hedatu** esaten zaio, eta objektuetara bideratutako programazioan oso maiz gertatuko den zerbait izango da.
- ▶ Bestalde, hemen funtzio bat dugu (**fun** hitz erreserbatua), funtzio horiek askatasunez egon daitezke eta ez dira inorenak edo klase batekoak izan daitezke guk klase baten barruan funtzioak definitzen ditugunean.

MAIN_ACTIVITY

- Klase bat zabaltzen dugunean eta aita klaseko funtzio batek egiten duena aldatu nahi dugunean, override hitz erreserbatua erabiltzen dugu, eta horrek esan nahi du aita klasea funtzio horretarako erabiltzen ari zen kodea berridatziko dugula.

```
</> class MainActivity : AppCompatActivity() {  
    @Override  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContentView(R.layout.activity_main)  
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->  
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())  
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)  
            insets  
        }  
    }  
}
```

MAIN_ACTIVITY

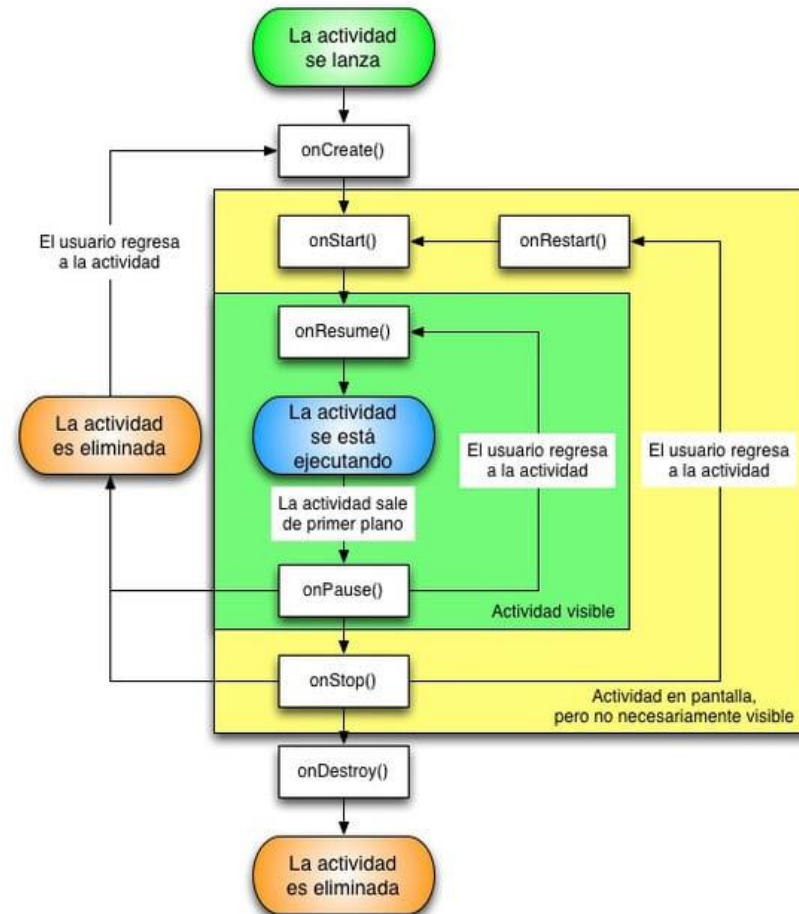
setContentView(R.layout.activity_main)

- ▶ Honek xml-a puztea esaten zaiona egingo du: xml testu-fitxategi bat besterik ez da, xml horretan erabiltzen ari garen bistei dagozkien klaseetako objektu serie bat bihurtzea da, guk gero kode bidez bista horietara sartu ahal izateko.
- ▶ Xml batetik bista bat berreskuratzeko egin behar dugun gauza bakarra findViewById funtzioa erabiltzea da, eta bista honetan jarri diogun R identifikatutik pasatzea.
- ▶ R Android klase berezi bat da, Android-eko baliabideen karpeta azpian biltzen diren baliabide guztiak batzen dituen.

MAIN_ACTIVITY

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val helloText = findViewById<TextView>(R.id.hello_text)  
        helloText.text = "Hello Android"  
    }  
}
```

ACTIVITY BATEN BIZI-ZIKLOA



ACTIVITY BATEN BIZI-ZIKLOA

Androiden bizi-zikloak une bakoitzean activity-ak duen ikusgarritasunarekin bat datozen urrats batzuk dira, hau da, jarduera erabat agerian dagoen, partzialki ikusgai dagoen, bigarren maila batera joan denean edo sistemak memoria-baliabideak behar dituelako suntsitzen ari den kontuan hartuta, gure activity-a hainbat puntutatik igaroko da.

► onCreate ()

Jarduera sortzen den unea adierazten du. Metodo hori, normalean, laguntzaileak sortuko du Androiden jarduera berri bat sortzean, eta hor sortuko dugu jarduerak beharko duen guztia. Oro har, hortxe egin ohi ditugu hasieraketa guztiak.

ACTIVITY BATEN BIZI-ZIKLOA

► onDestroy ()

Jarduera suntsitu egingo da eta baliabideak askatu. Zerbait desegin behar badugu jarduera suntsitu aurretik, puntu honetan burutzen da. Garrantzitsua da suntsitzea, adibidez, beste zerbitzu batzuetara konektatuta bagaude, gure gailuaren kokapena edo gauzak entzuten baditugu. Beraz, pisutsuagoak dira eta ez dugu nahi jarduera suntsitzen denean bizirik irautea.

► onStart ()

Jarduera pantailan egongo da, baina ez nahitaez ikusgai. Geltoki batetik bagatoz, aurretik onRestart-etik igaroko gara ().

ACTIVITY BATEN BIZI-ZIKLOA

- ▶ **onRestart ()**
onStart()-en aurrekoa, onStop()-era egindako dei batetik abiatzen garenean.
- ▶ **onResume()**
Jarduera erabiltzailearen interakzioari erantzuten hasiko da.
- ▶ **onPause ()**
Jarduerak erabiltzailearen interakzioari erantzuteari utziko dio.
- ▶ **onStop ()**
Jarduera bigarren mailara pasa da erabat.

ACTIVITY BATEN BIZI-ZIKLOA

► onCreate ()

Deiaren itzulketa hori inplementatu behar dugu, sistemak gure jarduera sortzen duenean aktibatzen dena. Gure inplementazioak hasiera eman behar die gure jardueraren funtsezko osagaiei. Adibidez, gure app-ak bistak sortu eta datuak zerrendetara lotu beharko lituzke hemen. Batez ere, une honetan `setContentView ()` deitu behar dugu gure jardueraren erabiltzaile-interfazearen diseinua definitzeko.

`onCreate ()` amaitzen denean, hurrengo deia itzultzea beti da `onStart ()`.

ACTIVITY BATEN BIZI-ZIKLOA

► onStart ()

onCreate () ixten denean, jarduera hasita egoerara pasatzen da eta erabiltzailearentzat ikusgai bihurtzen da. Deia itzultzeak jardueraren azken prestaketak ditu, lehen planora pasatzeko eta interaktibo bihurtzeko.

► onResume()

Sistemak dei hori itzultzea eskatzen du, jarduera erabiltzailearekin elkarreragiten hasi aurretik. Puntu honetan, jarduera jarduera-pilako lehena da, eta erabiltzaileak sartzen duen guztia harrapatzen du. App baten funtzionalitate nagusiaren zatirik handiena onResume() metodoan inplementatzen da.

Deia itzultzeko onPause () beti onResume() ondoren doa.

ACTIVITY BATEN BIZI-ZIKLOA

► onPause ()

Sistemak onPause () deitzen du jarduerak fokua galtzen duenean eta Geldi egoerara pasatzen denean. Egoera hori gertatzen da, adibidez, erabiltzaileak Atzera edo Berriak botoia sakatzen duenean. Sistemak gure jarduerarako onPause () deitzen duenean, jarduera oraindik partzialki ikusgai dagoela esan nahi du, baina, askotan, erabiltzailea jardueratik ateratzen ari dela eta laster Geldi edo Berrabiatua egoerara igaroko dela adierazten du. Geldi egoeran dagoen jarduera batek IU(Erabiltzailearen Interfazea) eguneratzen jarrai dezake erabiltzaileak ekintza hori gertatzea espero badu. Jarduera horren adibideen artean, nabigazio-maparen pantaila bat edo multimedia-aplikazioa baten erreprodukzioa agertzen dira. Jarduera horiek fokua galtzen badute ere, erabiltzaileak espero du IUk eguneratzen jarraitzea.

ACTIVITY BATEN BIZI-ZIKLOA

Ez erabili onPause () aplikazioaren edo erabiltzailearen datuak gordetzeko, sareko deiak egiteko edo datu-baseen transakzioak egiteko. Behin onPause () exekuzioa amaitzen denean, hurrengo dei-itzulketa onStop () edo onResume () izango da, jarduera geldi egoera pasatzen denean gertatzen denaren arabera.

ACTIVITY BATEN BIZI-ZIKLOA

► onStop ()

Sistemak onStop () deitzen du jarduera jada erabiltzailearentzat ikusgai ez dagoenean, eta hori gerta daiteke jarduera ezabatzen ari delako, berri bat hasten delako edo lehendik zegoen bat Berrekitua egoerara pasatzen delako eta gelditu zen jarduera estaltzen duelako. Kasu horietan guztietan, gelditutako jarduera jada ez dago batere agerian.

Sistemak aipatzen duen deiaren hurrengo itzulketa hau izan daiteke: [onRestart \(\)](#) (jarduerak erabiltzailearekin berriro elkarreragiten badu) edo [onDestroy \(\)](#) (jarduera erabat amaitzen bada)

ACTIVITY BATEN BIZI-ZIKLOA

► `onRestart()`

Sistemak deia itzultzea eskatzen du Geldi egoeran dagoen jarduera bat. `onRestart()`-ek jardueraren egoera berrezartzen du jarduera gelditu zen unetik. Deia itzuli ondoren, beti jarraitzen `onStart ()`.

► `onDestroy ()`

Sistemak deia itzultzea eskatzen du jarduera bat ezabatu baino lehen. Deia itzulketa hori da jarduera jasotzen duen azkena. `onDestroy ()` jarduera baten baliabide guztiak ezabatzen direnean edo jarduera hori duen prozesua ezabatzen denean askatzen direla bermatzeko implementatzen da.

ACTIVITY BATEN BIZI-ZIKLOA

- Log-ak erabili egoeretatik igarotzen ikusteko

